

Aula 7 — Temporizadores e Base de Tempo

Como um sistema passa a saber que horas são

Microcontroladores — DENE/UFMT

Objetivos da aula

- Explicar o funcionamento de um contador com divisor de entrada e distinguir os modos temporizador e contador.
- Calcular pré-carga e período para um intervalo desejado, e reconhecer as fontes de erro do resultado.
- Escolher entre os temporizadores disponíveis conforme a aplicação.
- Construir uma base de tempo de 1 ms e, sobre ela, um escalonador cooperativo capaz de executar tarefas em ritmos distintos.
- Distinguir deriva de flutuação e adotar a estratégia de recarga que evita a primeira.

1 O problema da espera ativa

Todo o firmware até aqui mediu tempo esperando. A função de atraso do compilador executa um laço vazio dimensionado para consumir a quantidade certa de ciclos — e, enquanto o consome, o processador não faz nada.

Atenção

Além do desperdício, há um problema de correção: o atraso por software supõe que nada interrompe o laço. Quando as interrupções entram em cena, cada interrupção atendida durante um atraso o **estende**, e o tempo real deixa de corresponder ao pedido. Atrasos por software e interrupções coexistem mal — outra razão para migrar para o temporizador.

Definição — temporizador

Contador em hardware que incrementa automaticamente a cada ciclo de uma fonte de clock, independentemente da execução do programa, e sinaliza quando transborda. No modo **contador**, a fonte é um sinal externo, e o mesmo periférico passa a contar eventos em vez de tempo.

A independência é a propriedade essencial: o contador avança enquanto o processador faz outra coisa, dorme ou atende interrupções. O tempo deixa de ser consumido e passa a ser **observado**.

2 Os temporizadores do PIC18F4550

Módulo	Bits	Característica	Uso típico
Temporizador 0	8 ou 16	Divisor de entrada amplo, largura selecionável	Base de tempo geral

Módulo	Bits	Característica	Uso típico
Temporizador 1	16	Aceita cristal externo de baixa frequência	Relógio de tempo real; despertar do modo de baixo consumo
Temporizador 2	8	Possui registrador de período e divisor de saída	Base de tempo exata; obrigatório para a modulação por largura de pulso
Temporizador 3	16	Semelhante ao 1	Captura e comparação

Tabela 1: Os quatro temporizadores e suas vocações.

Observação

A terceira linha merece destaque porque decide o encontro seguinte: o temporizador 2 é o único com **registrador de período**, isto é, com um valor de comparação que reinicia a contagem automaticamente. Isso o torna exato — e é a razão de o módulo de modulação por largura de pulso depender obrigatoriamente dele.

3 Cálculo do intervalo

Derivação

O contador avança a cada ciclo de instrução dividido pelo fator do divisor de entrada. O intervalo até o transbordo, partindo de um valor de pré-carga P num contador de n bits, é

$$t = T_{cy} \cdot D \cdot (2^n - P)$$

em que D é o fator do divisor. Isolando a pré-carga:

$$P = 2^n - \frac{t}{T_{cy} \cdot D}$$

Exemplo

Objetivo: 1 ms com oscilador de 48 MHz, contador de 16 bits e divisor 1:16. Com $T_{cy} \approx 83,33$ ns, cada incremento vale

$$83,33 \text{ ns} \cdot 16 \approx 1,333 \text{ } \mu\text{s}$$

O número de incrementos necessários é

$$1 \frac{\text{ms}}{1,333 \text{ } \mu\text{s}} = 750$$

e a pré-carga vale

$$P = 65536 - 750 = 64786$$

Atenção

A conta acima só fecha quando o número de incrementos resulta inteiro. Se resultar fracionário, o intervalo obtido difere do pedido e o erro se acumula a cada ciclo — em um

relógio, essa fração vira minutos ao fim de um dia. Nesses casos, ou se ajusta o divisor, ou se escolhe um intervalo vizinho que feche exato, ou se compensa periodicamente em software.

4 Deriva e flutuação

Dois erros distintos afetam uma base de tempo, e confundi-los leva a correções inúteis.

Definição — deriva e flutuação

Deriva é o erro sistemático acumulado: cada ciclo é ligeiramente mais longo que o pretendido, e o desvio cresce indefinidamente. **Flutuação** é a variação aleatória em torno do valor correto, sem acúmulo.

Derivação

Considere o tratador que recarrega o contador escrevendo o valor absoluto de pré-carga. Entre o transbordo e a escrita decorrem a latência de interrupção e as instruções iniciais do tratador — digamos t_e . Como a contagem recomeça do valor escrito, e não da posição em que estava, esses t_e são **perdidos** a cada ciclo:

$$t_{\text{real}} = t_{\text{desejado}} + t_e$$

Com t_e de 2 μs e período de 1 ms, o erro é de 0,2%: dezessete segundos por dia.

Código

```
/* Recarga por soma: preserva o excedente da contagem
   e elimina a deriva sistematica. */
void __interrupt(high_priority) isr_alta(void)
{
    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;

        /* soma a precarga ao valor atual, em vez de sobrescrever */
        uint16_t atual = (uint16_t)(((uint16_t)TMR0H << 8) | TMR0L);
        atual += PRECARGA;
        TMR0H = (uint8_t)(atual >> 8);
        TMR0L = (uint8_t)(atual & 0xFF);

        g_tique = 1;
    }
}
```

Atenção

A ordem de escrita dos dois bytes do contador não é indiferente: a parte alta é armazenada num registrador intermediário e só é efetivada quando a parte baixa é escrita. **Escrever na ordem errada corrompe a contagem**, e o mesmo vale para a leitura, que deve começar pela parte baixa. É um mecanismo de atomicidade em hardware, análogo à discussão de acesso não atômico do encontro 9 — e vale conferir a ordem exigida na folha de dados.

Observação

A solução definitiva contra a deriva é o registrador de período do temporizador 2, que recarrega em hardware, sem participação do software e portanto sem latência alguma. Quando a exatidão importa, esse é o módulo a usar — e é essa a razão de ele ser a base do encontro seguinte.

5 O escalonador cooperativo

Com uma base de 1 ms, o sistema ganha uma noção de tempo, e as tarefas passam a ser expressas em múltiplos dela.

Código

```
volatile uint8_t g_tique = 0;

int main(void)
{
    uint16_t ms_display = 0, ms_sensor = 0, ms_botao = 0;

    sistema_iniciar();

    for (;;) {
        if (!g_tique) {
            continue;
        }
        g_tique = 0;                                /* consome o tique */

        if (++ms_botao >= 5u) {                       /* a cada 5 ms */
            ms_botao = 0;
            botao_mostrar();
        }

        if (++ms_sensor >= 100u) {                   /* a cada 100 ms */
            ms_sensor = 0;
            temperatura = adc_ler(CANAL_LM35);
            controlar(temperatura, alvo);
        }

        if (++ms_display >= 300u) {                 /* a cada 300 ms */
            ms_display = 0;
            atualizar_display(temperatura);
        }
    }
}
```

Observação

Três ritmos distintos convivem sem que nenhum bloqueie os demais, e a estrutura não muda quando se acrescenta a quarta tarefa. Compare com a versão em varredura do encontro 2: lá, cada funcionalidade nova degradava o tempo de resposta de todas as anteriores. **Este é**

o ponto de virada arquitetural do curso — daqui em diante o firmware só cresce em largura, nunca em profundidade.

Atenção

A palavra **cooperativo** é literal: nenhuma tarefa é interrompida por outra, de modo que cada uma precisa devolver o controle rapidamente. Uma tarefa que demore mais que o período do tique faz os tiques seguintes serem perdidos — e o sistema passa a marcar o tempo mais devagar que o relógio. A escrita no display, com 1,3 ms, está justamente nessa fronteira e merece medição.

6 Transposição

Todo microcontrolador de 32 bits traz um temporizador dedicado à base de tempo do sistema, com finalidade idêntica à construída aqui. As diferenças são convenientes: contadores de 32 bits eliminam boa parte dos problemas de faixa, e o registrador de recarga automática elimina a deriva por construção — não há software na malha de recarga.

Observação

O escalonador cooperativo desta aula é, também, o degrau anterior ao sistema operacional de tempo real. Um sistema desses substitui o encadeamento de contadores por tarefas independentes com suas próprias pilhas, capazes de suspender e retomar. A troca é clara: ganha-se organização e preempção, paga-se em memória e em previsibilidade. Para o projeto deste semestre, o escalonador cooperativo é a escolha correta — e reconhecer **quando** deixaria de ser é parte do que se avalia no seminário.

7 Exercícios

Exercício 7.1

Calcule a pré-carga para obter 10 ms com oscilador de 20 MHz, contador de 16 bits e divisor 1:32. Verifique se o número de incrementos é inteiro e, se não for, proponha um ajuste.

Exercício 7.2

Um sistema usa recarga por sobrescrita com $t_e = 3 \mu\text{s}$ e período nominal de 2 ms. Calcule o erro percentual, o atraso acumulado em 24 h e o número de tiques perdidos nesse intervalo.

Exercício 7.3

Acrescente ao escalonador da seção 5 uma quarta tarefa que envie telemetria a cada 2 s, sem alterar a estrutura existente. Justifique o tipo escolhido para o contador correspondente.

Exercício 7.4

Explique por que o temporizador 2 é preferível ao 0 quando a exatidão do período é crítica, apontando o mecanismo de hardware responsável pela diferença.

Exercício 7.5

Um estudante afirma que o escalonador cooperativo “não é tempo real, porque uma tarefa longa atrasa as outras”. Avalie a afirmação: em que sentido ela está correta, em que sentido é imprecisa, e o que exatamente seria necessário para caracterizar o sistema como de tempo real.